

Architecture Overview

Comprehensive Data Ingestion Architecture Guide

Table of Contents

- 1. [Introduction](#)
- 2. [Architectural Layers](#)
- 3. [Architecture Patterns](#)
- 4. [Batch vs Streaming vs Real-Time](#)
- 5. [Lambda Architecture](#)
- 6. [Kappa Architecture](#)
- 7. [Microservices-Based Ingestion](#)
- 8. [Cloud-Native Architectures](#)
- 9. [Reference Architectures by Scale](#)
- 10. [Component Interactions](#)
- 11. [Network Topology](#)
- 12. [High Availability & Fault Tolerance](#)

1. Introduction {#introduction}

What is Data Ingestion?

Data ingestion is the process of obtaining, importing, and processing data from various sources into a storage system where it can be accessed, analyzed, and utilized by downstream applications and analytics tools.

Key Objectives

Primary Goals:

- **Reliability:** No data loss, guaranteed delivery
- **Scalability:** Handle growing data volumes
- **Low Latency:** Minimize time from source to destination
- **Data Quality:** Ensure accuracy, completeness, consistency
- **Cost Efficiency:** Optimize resource utilization

Secondary Goals:

- **Flexibility:** Support diverse data sources and formats
- **Observability:** Monitor and troubleshoot easily
- **Security:** Protect data in transit and at rest
- **Governance:** Track lineage, ensure compliance

The Evolution of Data Ingestion

- 1990s: ETL Batch Processing
 - └ Nightly batch jobs
 - └ Data warehouses (Oracle, Teradata)
 - └ Hours of latency acceptable
- 2000s: Near Real-Time ETL
 - └ Change Data Capture (CDC)
 - └ Micro-batching (15-min windows)
 - └ Minutes of latency
- 2010s: Stream Processing
 - └ Apache Kafka, Storm, Spark Streaming
 - └ Event-driven architectures
 - └ Seconds of latency
- 2020s: Real-Time + Serverless
 - └ Apache Flink, Pulsar
 - └ Cloud-native, serverless functions
 - └ Sub-second latency
 - └ Exactly-once semantics

2. Architectural Layers {#architectural-layers}

Modern Data Ingestion Stack (7 Layers)

Layer 7: Consumption Layer
(BI Tools, ML Models, Applications)

↑

Layer 6: Serving Layer
(Data Warehouse, OLAP, Feature Store)

↑

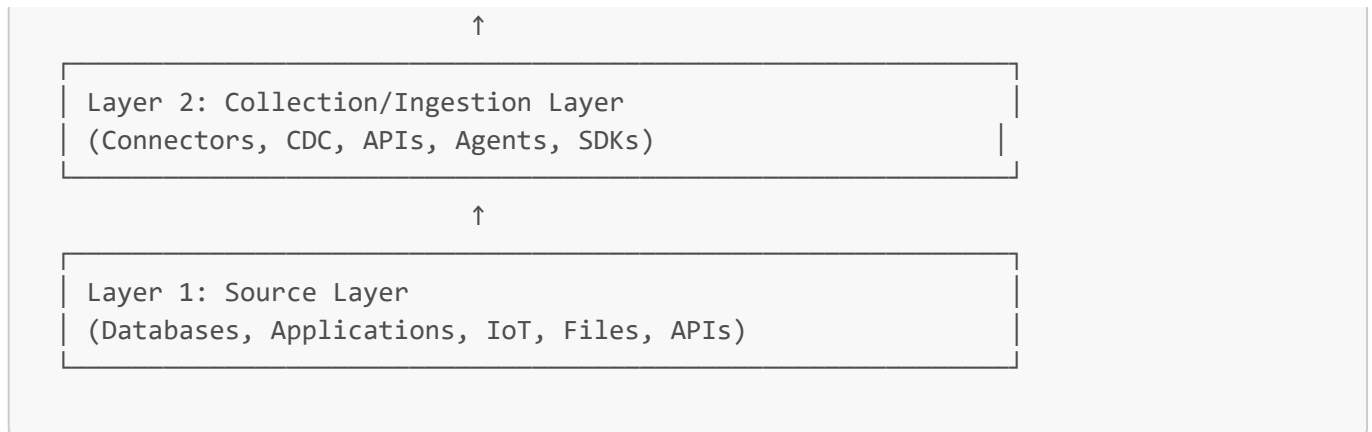
Layer 5: Storage Layer
(Data Lake, Object Storage, Distributed FS)

↑

Layer 4: Processing Layer
(Stream Processing, Batch Processing, Transformation)

↑

Layer 3: Message/Buffer Layer
(Kafka, Kinesis, Pub/Sub, Event Hubs)



Layer 1: Source Layer

Data Sources:

Structured Data:

- Relational databases (MySQL, PostgreSQL, Oracle, SQL Server)
- NoSQL databases (MongoDB, Cassandra, DynamoDB)
- SaaS applications (Salesforce, Zendesk, HubSpot)

Semi-Structured Data:

- JSON APIs (REST, GraphQL)
- XML feeds
- Log files (application logs, web server logs)
- Event streams (clickstream, user activity)

Unstructured Data:

- Files (CSV, Parquet, Avro, ORC)
- Object storage (images, videos, documents)
- Social media feeds
- Email

IoT & Sensor Data:

- Industrial sensors (temperature, pressure, vibration)
- GPS devices
- Smart devices
- Medical equipment

Characteristics:

- **Volume:** TB to PB per day
- **Velocity:** Real-time to batch
- **Variety:** Multiple formats and schemas
- **Veracity:** Data quality varies

Layer 2: Collection/Ingestion Layer

Components:

1. Database Connectors

Purpose: Extract data from databases

Technologies:

- Debezium (CDC for MySQL, PostgreSQL, MongoDB, etc.)
- Oracle GoldenGate
- AWS DMS (Database Migration Service)
- Attunity Replicate

Features:

- Change Data Capture (CDC)
- Full and incremental loads
- Schema evolution handling
- Heartbeat monitoring

2. API Connectors

Purpose: Pull data from REST/GraphQL APIs

Technologies:

- Airbyte
- Fivetran
- Stitch
- Custom connectors (Python, Java)

Features:

- Rate limiting
- Pagination handling
- Authentication (OAuth, API keys)
- Error handling and retries

3. File Ingestion Agents

Purpose: Monitor and ingest files

Technologies:

- Apache NiFi
- Fluentd
- Filebeat
- AWS S3 Event Notifications

Features:

- File watching (inotify, S3 events)
- Format detection
- Decompression
- Checksumming

4. Stream Producers

Purpose: Publish events to message brokers

Technologies:

- Kafka Producer API
- Kinesis Producer Library (KPL)
- SDK clients (Python, Java, Go)

Features:

- Batching
- Compression
- Partitioning
- Acknowledgments

5. SDKs & Instrumentation

Purpose: Application-level data collection

Technologies:

- Segment
- Snowplow
- Custom SDKs

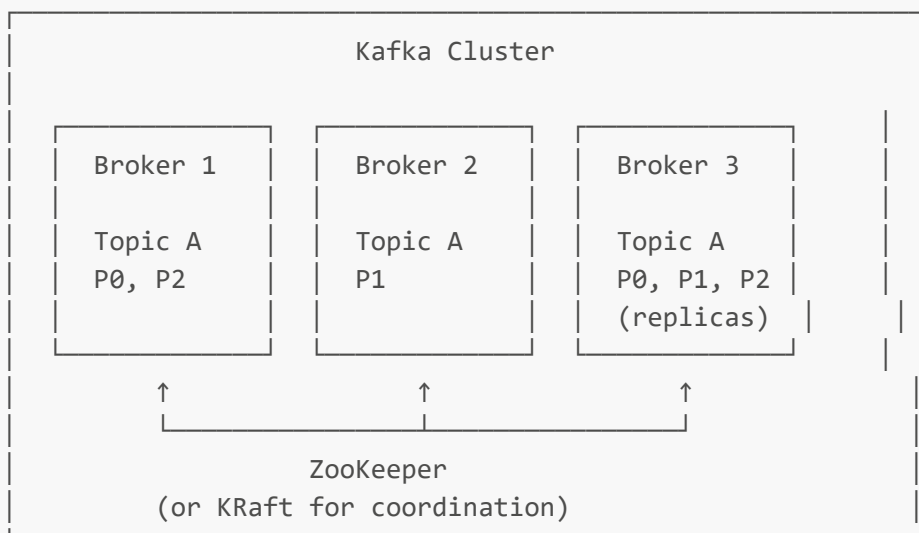
Features:

- Client-side (JavaScript, mobile)
- Server-side (backend services)
- Auto-tracking (page views, events)

Layer 3: Message/Buffer Layer

Purpose: Decouple producers from consumers, provide buffering, enable stream processing

Apache Kafka Architecture:



↑
Producers
(Write events)

↓
Consumers
(Read events)

Key Concepts:

Topics & Partitions:

Topic: "user-events"

- | Partition 0: [event1, event3, event5, ...]
- | Partition 1: [event2, event6, event9, ...]
- | Partition 2: [event4, event7, event8, ...]

Benefits:

- Parallelism: Multiple consumers read different partitions
- Ordering: Within-partition ordering guaranteed
- Scalability: Add partitions to increase throughput

Replication:

Topic: "transactions" (Replication Factor = 3)

Broker 1 (Leader for P0): [Partition 0]
 Broker 2 (Follower for P0): [Partition 0] (replica)
 Broker 3 (Follower for P0): [Partition 0] (replica)

If Broker 1 fails:

- Broker 2 or 3 becomes leader
- No data loss
- Producers/consumers automatically reconnect

Consumer Groups:

Consumer Group: "analytics-processors"

- | Consumer 1 → reads from Partition 0, 1
- | Consumer 2 → reads from Partition 2, 3
- | Consumer 3 → reads from Partition 4, 5

Consumer Group: "real-time-alerts"

- | Consumer 1 → reads from ALL partitions (independent)

Benefits:

- Multiple applications consume same data
- Load balancing within group
- Fault tolerance (rebalancing)

Retention & Compaction:

Time-based Retention:

- Keep messages for 7 days
- Delete older messages
- Use case: Log aggregation

Size-based Retention:

- Keep 100 GB per partition
- Delete oldest when limit reached
- Use case: High-volume streams

Log Compaction:

- Keep only latest value per key
- Use case: Database CDC (maintain latest state)

Example:

Before compaction:

```
key=user_123, value={name: "Alice"}    (timestamp: T1)
key=user_123, value={name: "Alice", age: 30}  (T2)
key=user_123, value={name: "Alice", age: 31}  (T3)
```

After compaction:

```
key=user_123, value={name: "Alice", age: 31}  (T3)
```

Alternative Technologies:

AWS Kinesis Data Streams:

Architecture:

- Streams divided into shards (not partitions)
- Each shard: 1 MB/s write, 2 MB/s read
- Managed service (no broker management)

Pros:

- Serverless (AWS managed)
- Integrated with AWS ecosystem
- Simple to set up

Cons:

- Shard management complexity
- More expensive at scale
- 1 MB message size limit

Google Cloud Pub/Sub:

Architecture:

- Topics and subscriptions (push/pull)

- Auto-scaling (no partition management)
- Global distribution

Pros:

- Fully managed, serverless
- Exactly-once delivery
- At-least-once by default

Cons:

- No ordering guarantees (without ordering key)
- Message retention: 7 days max

Apache Pulsar:**Architecture:**

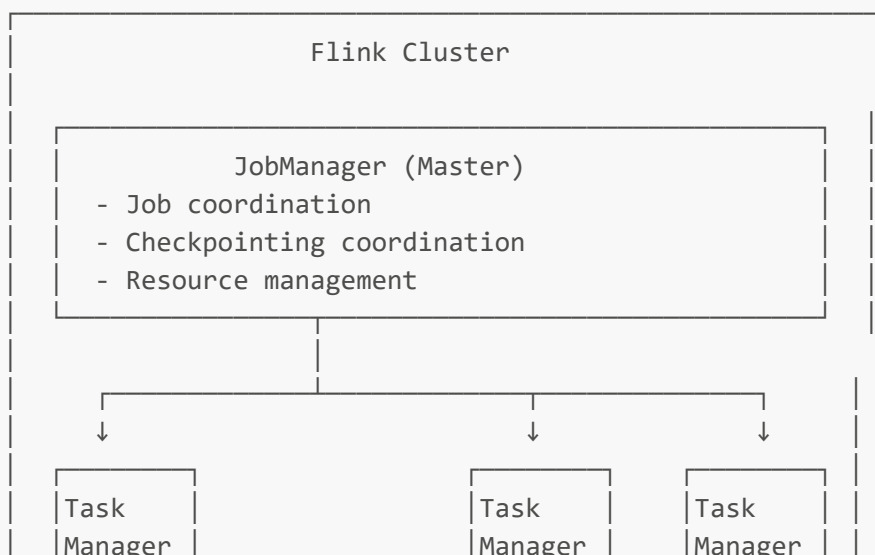
- Separates compute (brokers) from storage (BookKeeper)
- Multi-tenancy built-in
- Geo-replication

Pros:

- Better isolation (topics don't share resources)
- Infinite retention (tiered storage)
- Built-in multi-DC replication

Cons:

- Smaller ecosystem than Kafka
- More complex architecture

Layer 4: Processing Layer**Stream Processing:****Apache Flink Architecture:**



Flink Programming Model:

```
// Flink job for real-time aggregation
StreamExecutionEnvironment env =
    StreamExecutionEnvironment.getExecutionEnvironment();

// Enable checkpointing (for fault tolerance)
env.enableCheckpointing(60000); // checkpoint every 60 seconds

// Source: Read from Kafka
FlinkKafkaConsumer<String> consumer = new FlinkKafkaConsumer<>(
    "user-events",
    new SimpleStringSchema(),
    kafkaProps
);

DataStream<UserEvent> events = env
    .addSource(consumer)
    .map(json -> parseUserEvent(json))
    .assignTimestampsAndWatermarks(
        WatermarkStrategy
            .<UserEvent>forBoundedOutOfOrderness(Duration.ofSeconds(10))
            .withTimestampAssigner((event, timestamp) -> event.timestamp)
    );

// Transformation: Window and aggregate
DataStream<UserMetrics> metrics = events
    .keyBy(event -> event.userId)
    .window(TumblingEventTimeWindows.of(Time.minutes(5)))
    .aggregate(new CountAggregator());

// Sink: Write to data lake
metrics.addSink(new S3Sink<>(
    "s3://data-lake/user-metrics/",
    new ParquetWriter()
));

env.execute("User Metrics Pipeline");
```

Key Flink Features:

1. Event Time Processing:

Event Time vs Processing Time

Event Time: When event actually occurred

Processing Time: When system processes event

Example:

Event generated: 10:00:00 AM (event time)

Network delay: 2 minutes

Event processed: 10:02:00 AM (processing time)

Why Event Time Matters:

- Reprocessing gives same results
- Handles out-of-order events
- Late data can be incorporated

2. Watermarks:

Watermark: Signal indicating "all events before time T have arrived"

Example:

Events stream in:

t=100, t=102, t=98, t=105, t=103, ...

Watermark at t=100 means:

"No more events with timestamp < 100 will arrive"

Bounded Out-of-Orderness:

Allow 10 seconds of delay

Watermark = max_event_time - 10 seconds

3. State Management:

```
// Keyed State (per-key state)
class CountingFunction extends RichFlatMapFunction<Event, Result> {
    private ValueState<Long> countState;

    @Override
    public void open(Configuration config) {
        ValueStateDescriptor<Long> descriptor =
            new ValueStateDescriptor<>("count", Long.class, 0L);
        countState = getRuntimeContext().getState(descriptor);
    }

    @Override
    public void flatMap(Event event, Collector<Result> out) {
        long currentCount = countState.value();
        currentCount++;
        countState.update(currentCount);
    }
}
```

```

        if (currentCount >= 100) {
            out.collect(new Result(event.key, currentCount));
            countState.clear();
        }
    }
}

// State stored in RocksDB (embedded key-value store)
// Checkpointed to S3/HDFS for fault tolerance

```

4. Exactly-Once Processing:

Two-Phase Commit Protocol:

Phase 1: Pre-commit

- | Flink checkpoint starts
- | Stop processing new records
- | Flush all buffered data
- | Pre-commit to external systems (Kafka, database)
- | Checkpoint state to S3

Phase 2: Commit

- | Checkpoint successful
- | Commit transactions to external systems
- | Resume processing

If failure occurs:

- | Rollback to last checkpoint
- | Replay records from Kafka (offset stored in checkpoint)
- | External systems rollback pre-commits

Result: Each record processed exactly once

Apache Spark Structured Streaming:

```

// Spark Structured Streaming job
val spark = SparkSession.builder()
    .appName("UserMetrics")
    .getOrCreate()

// Read from Kafka
val events = spark
    .readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "localhost:9092")
    .option("subscribe", "user-events")
    .load()
    .selectExpr("CAST(value AS STRING)")
    .select(from_json($"value", eventSchema).as("data"))

```

```
.select("data.*")

// Transformation with windowing
val metrics = events
  .withWatermark("timestamp", "10 minutes")
  .groupBy(
    window($"timestamp", "5 minutes"),
    $"userId"
  )
  .agg(
    count("*").as("event_count"),
    countDistinct("eventType").as("unique_events")
  )

// Write to Delta Lake
val query = metrics
  .writeStream
  .format("delta")
  .outputMode("append")
  .option("checkpointLocation", "/tmp/checkpoint")
  .start("/data/user-metrics")

query.awaitTermination()
```

Spark vs Flink Comparison:

Feature	Apache Flink	Apache Spark Streaming
Processing Model	True streaming (event-by-event)	Micro-batching (small batches)
Latency	Sub-second (100ms)	1-2 seconds
Throughput	High	Very high
State Management	Built-in, efficient (RocksDB)	More limited
Exactly-Once	Native support	Supported (with idempotent sinks)
SQL Support	Flink SQL	Spark SQL (more mature)
Ecosystem	Smaller	Large (MLlib, GraphX)
Ease of Use	Moderate	Easier (more examples)
Batch + Stream	Unified (DataStream API)	Unified (DataFrame API)

Batch Processing:

Apache Spark (Batch):

```
# PySpark batch job
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
```

```

spark = SparkSession.builder \
    .appName("DailyMetrics") \
    .getOrCreate()

# Read from S3 (Parquet files)
events = spark.read.parquet("s3://raw-data/events/date=2026-08-15/")

# Transformations
user_metrics = events \
    .groupBy("user_id") \
    .agg(
        count("*").alias("total_events"),
        countDistinct("session_id").alias("sessions"),
        sum("revenue").alias("total_revenue")
    ) \
    .withColumn("avg_revenue_per_session",
        col("total_revenue") / col("sessions"))

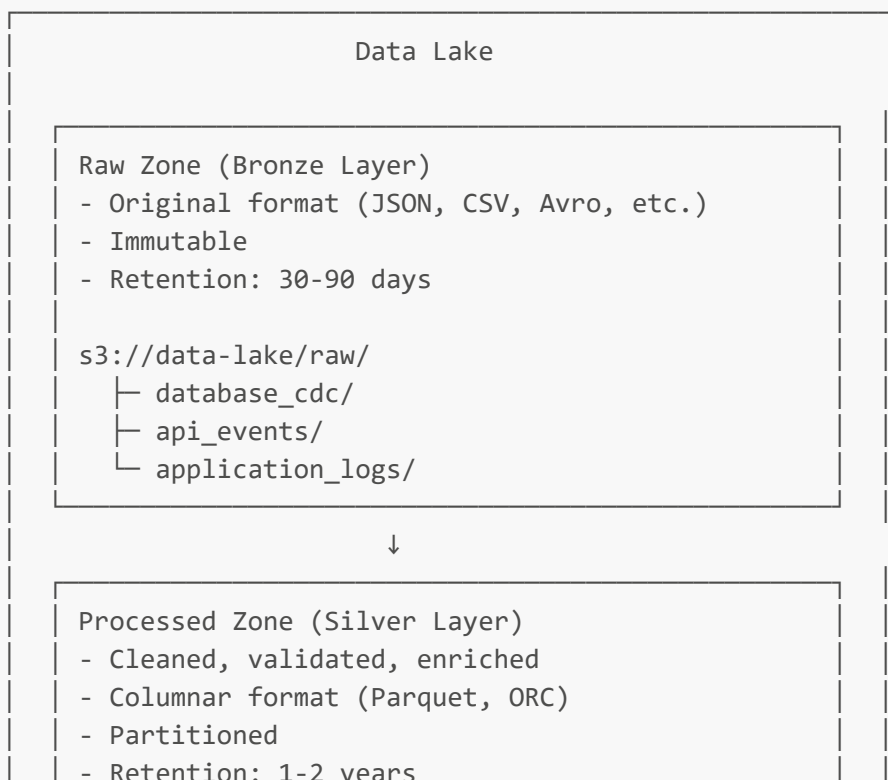
# Write to data warehouse
user_metrics.write \
    .mode("overwrite") \
    .partitionBy("date") \
    .parquet("s3://processed-data/user-metrics/")

spark.stop()

```

Layer 5: Storage Layer

Data Lake Architecture:





File Formats Comparison:

Format	Type	Compression	Splittable	Schema Evolution	Best For
Parquet	Columnar	Excellent	Yes	Good	Analytics, OLAP
ORC	Columnar	Excellent	Yes	Good	Hive, Presto
Avro	Row-based	Good	Yes	Excellent	Streaming, CDC
JSON	Row-based	Fair	No	None	Raw data, logs
CSV	Row-based	Fair	Yes	None	Simple datasets

Partitioning Strategy:

```
# Good partitioning (reduces data scanned)
s3://data-lake/events/
  year=2024/
    month=01/
      day=15/
        hour=10/
          part-00000.parquet
          part-00001.parquet

Query: SELECT * FROM events WHERE year=2024 AND month=01 AND day=15
Result: Scans only 1 day of data (1/365 of dataset)

# Bad partitioning (too granular)
s3://data-lake/events/
  user_id=user_12345/
    event_type=click/
```

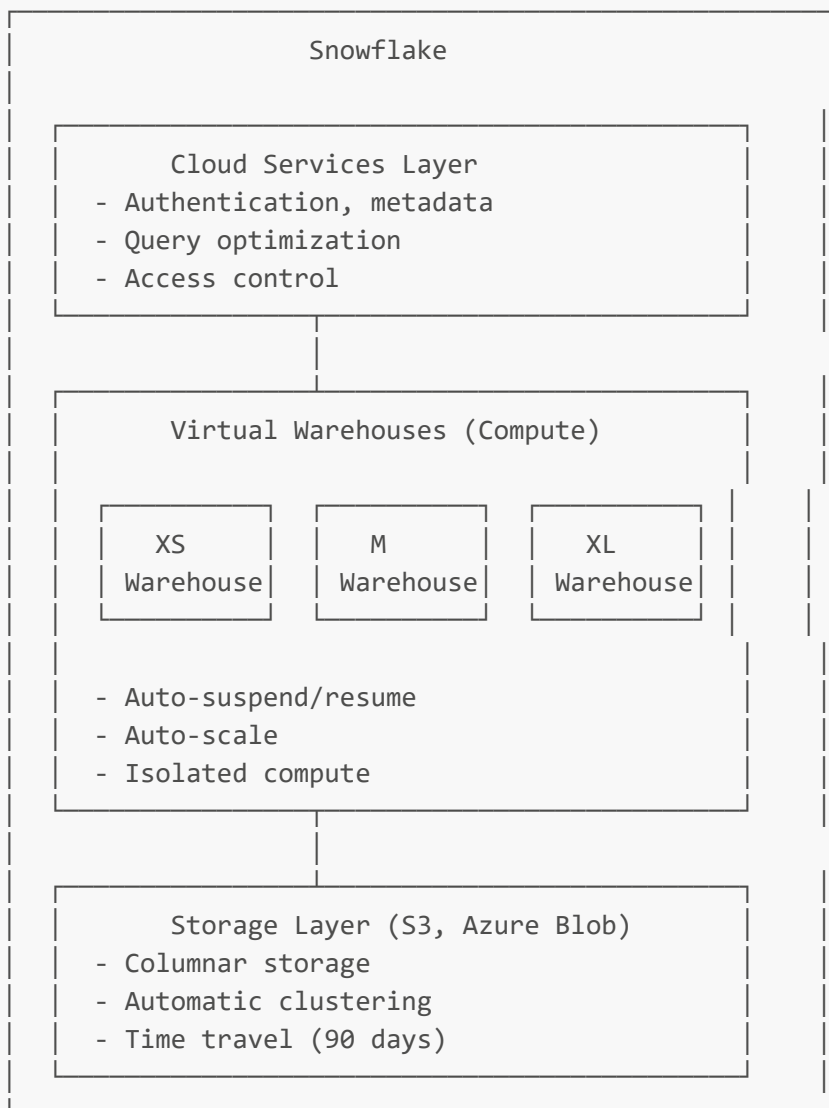
```
timestamp=2026-08-15-10-30-45/
data.parquet
```

Problem: Millions of tiny files, expensive metadata operations

Layer 6: Serving Layer

Data Warehouse:

Snowflake Architecture:



Loading Data into Warehouse:

```
-- Snowflake: Load from S3
COPY INTO user_metrics
FROM s3://data-lake/processed/user-metrics/
CREDENTIALS = (AWS_KEY_ID='xxx' AWS_SECRET_KEY='yyy')
FILE_FORMAT = (TYPE = PARQUET)
```

```
PATTERN = '.*'/year=2024/month=01/.*';

-- Incremental load with Snowpipe (auto-ingest)
CREATE PIPE user_metrics_pipe AS
COPY INTO user_metrics
FROM s3://data-lake/processed/user-metrics/
FILE_FORMAT = (TYPE = PARQUET);

-- Configure S3 event notification to trigger Snowpipe
ALTER PIPE user_metrics_pipe REFRESH;
```

Layer 7: Consumption Layer

BI Tools:

- Tableau, Power BI, Looker
- Connect to data warehouse
- Interactive dashboards

ML Models:

- Read from feature stores
- Batch predictions (Spark)
- Real-time predictions (Flink, API)

Applications:

- APIs serving data
- Real-time dashboards
- Operational applications

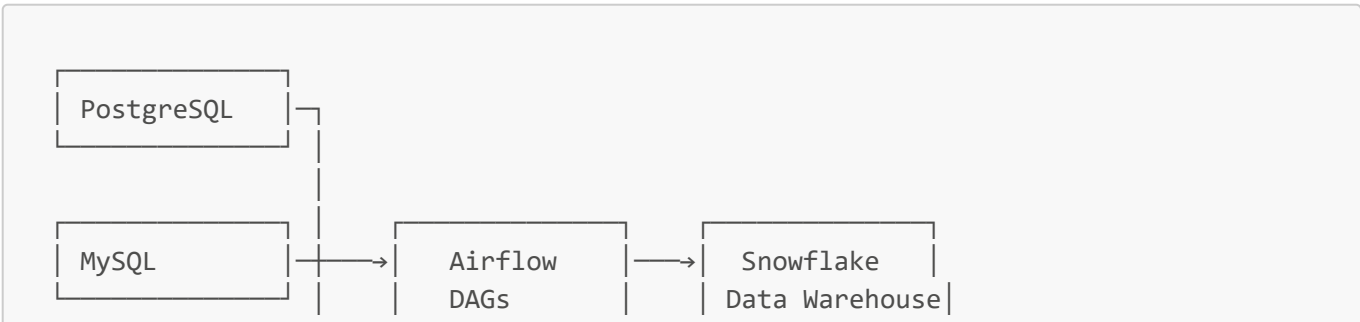
3. Architecture Patterns {#architecture-patterns}

Pattern 1: Simple Batch ETL

Use Case: Small to medium data volumes, latency tolerance > 1 hour

```
Sources → Airflow → Transform → Load → Data Warehouse
```

Architecture Diagram:





API (REST)

Schedule: Nightly at 2 AM

Duration: 2-4 hours

Volume: <100 GB/day

Airflow DAG Example:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.postgres.hooks.postgres import PostgresHook
from airflow.providers.snowflake.hooks.snowflake import SnowflakeHook
from datetime import datetime, timedelta
import pandas as pd

default_args = {
    'owner': 'data-eng',
    'depends_on_past': False,
    'email_on_failure': True,
    'email_on_retry': False,
    'retries': 3,
    'retry_delay': timedelta(minutes=5),
}

dag = DAG(
    'daily_etl_pipeline',
    default_args=default_args,
    description='Daily ETL from PostgreSQL to Snowflake',
    schedule_interval='0 2 * * *', # 2 AM daily
    start_date=datetime(2024, 1, 1),
    catchup=False,
)

def extract_from_postgres(**context):
    """Extract data from PostgreSQL"""
    execution_date = context['execution_date']
    pg_hook = PostgresHook(postgres_conn_id='postgres_db')

    query = f"""
    SELECT * FROM transactions
    WHERE created_at >= '{execution_date - timedelta(days=1)}'
    AND created_at < '{execution_date}'
    """

    df = pg_hook.get_pandas_df(query)

    # Save to temp location
    df.to_parquet(f'/tmp/transactions_{execution_date.date()}.parquet')
    return f'/tmp/transactions_{execution_date.date()}.parquet'
```

```

def transform(**context):
    """Transform data"""
    file_path = context['task_instance'].xcom_pull(task_ids='extract')
    df = pd.read_parquet(file_path)

    # Transformations
    df['amount_usd'] = df['amount'] * df['exchange_rate']
    df['transaction_hour'] = pd.to_datetime(df['created_at']).dt.hour
    df['is_weekend'] = pd.to_datetime(df['created_at']).dt.dayofweek >= 5

    # Data quality checks
    assert df['amount_usd'].notna().all(), "Null values in amount_usd"
    assert (df['amount_usd'] >= 0).all(), "Negative amounts detected"

    transformed_path = file_path.replace('.parquet', '_transformed.parquet')
    df.to_parquet(transformed_path)
    return transformed_path

def load_to_snowflake(**context):
    """Load data to Snowflake"""
    file_path = context['task_instance'].xcom_pull(task_ids='transform')
    execution_date = context['execution_date']

    snowflake_hook = SnowflakeHook(snowflake_conn_id='snowflake_dw')

    # Upload to S3 staging
    s3_path = f"s3://staging-bucket/transactions/{execution_date.date()}.parquet"
    upload_to_s3(file_path, s3_path)

    # Load into Snowflake
    sql = f"""
    COPY INTO analytics.transactions
    FROM '{s3_path}'
    FILE_FORMAT = (TYPE = PARQUET)
    """

    snowflake_hook.run(sql)

# Define task dependencies
extract_task = PythonOperator(
    task_id='extract',
    python_callable=extract_from_postgres,
    dag=dag,
)

transform_task = PythonOperator(
    task_id='transform',
    python_callable=transform,
    dag=dag,
)

load_task = PythonOperator(
    task_id='load',

```

```
python_callable=load_to_snowflake,  
dag=dag,  
)  
  
# Set dependencies  
extract_task >> transform_task >> load_task
```

Pros:

- Simple to implement and understand
- Predictable resource usage
- Easy to troubleshoot

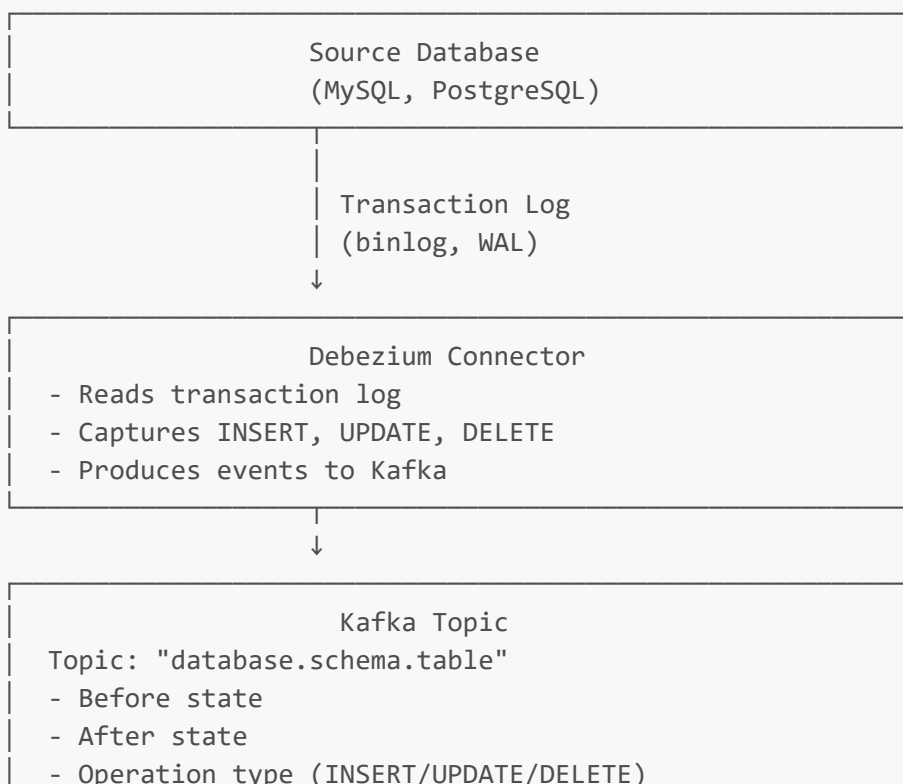
Cons:

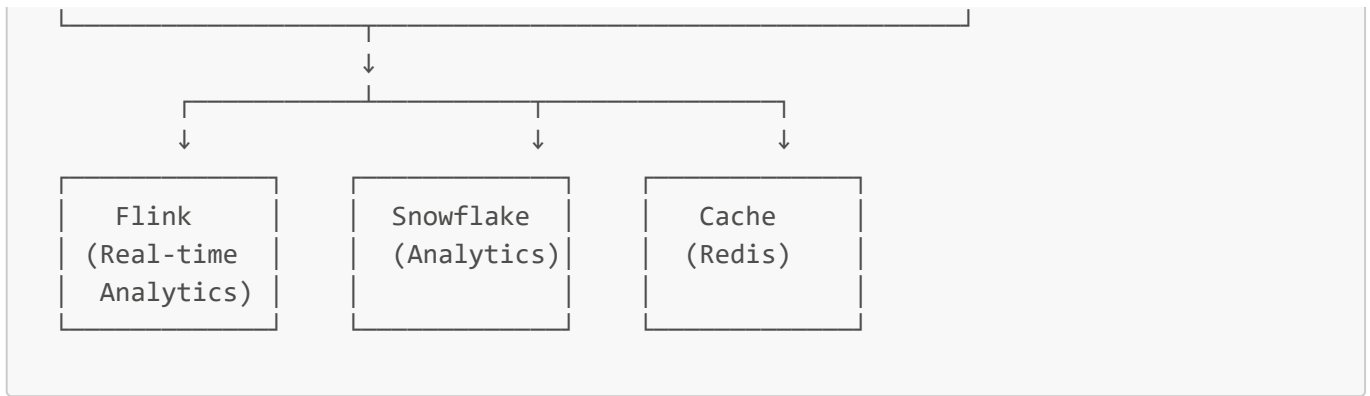
- High latency (hours)
- Not suitable for real-time use cases
- Full scans can be expensive

Pattern 2: Change Data Capture (CDC) Streaming

Use Case: Real-time database replication, event-driven architecture

Database → Debezium → Kafka → Stream Processor → Data Lake/DW

Architecture Diagram:



Debezium Configuration (MySQL CDC):

```
{
  "name": "mysql-connector",
  "config": {
    "connector.class": "io.debezium.connector.mysql.MySqlConnector",
    "database.hostname": "mysql.example.com",
    "database.port": "3306",
    "database.user": "debezium",
    "database.password": "password",
    "database.server.id": "184054",
    "database.server.name": "production",
    "database.include.list": "ecommerce",
    "table.include.list": "ecommerce.orders,ecommerce.order_items",
    "database.history.kafka.bootstrap.servers": "kafka:9092",
    "database.history.kafka.topic": "schema-changes",
    "include.schema.changes": "true",
    "transforms": "unwrap",
    "transforms.unwrap.type": "io.debezium.transforms.ExtractNewRecordState",
    "transforms.unwrap.drop.tombstones": "false",
    "transforms.unwrap.delete.handling.mode": "rewrite",
    "snapshot.mode": "initial",
    "snapshot.locking.mode": "minimal"
  }
}
```

CDC Event Structure:

```
{
  "before": null,
  "after": {
    "order_id": 12345,
    "user_id": 789,
    "amount": 99.99,
    "status": "pending",
    "created_at": "2026-08-15T10:30:00Z"
  },
  "source": {
    "version": "2.1.0",
  }
}
```

```

    "connector": "mysql",
    "name": "production",
    "ts_ms": 1705315800000,
    "snapshot": "false",
    "db": "ecommerce",
    "table": "orders",
    "server_id": 184054,
    "gtid": null,
    "file": "mysql-bin.000003",
    "pos": 154,
    "row": 0,
    "thread": 7,
    "query": null
  },
  "op": "c",
  "ts_ms": 1705315800123,
  "transaction": null
}

```

Processing CDC Events with Flink:

```

// Flink job to process CDC events
StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();

// Source: Kafka topic with CDC events
FlinkKafkaConsumer<String> consumer = new FlinkKafkaConsumer<>(
    "production.ecommerce.orders",
    new SimpleStringSchema(),
    kafkaProperties
);

DataStream<OrderEvent> orders = env
    .addSource(consumer)
    .map(new CDCDeserializer())
    .filter(event -> event.op.equals("c") || event.op.equals("u")); // INSERT or
UPDATE only

// Transform and enrich
DataStream<EnrichedOrder> enriched = orders
    .keyBy(order -> order.userId)
    .process(new UserEnrichmentFunction()); // Lookup user details

// Sink to multiple destinations
enriched.addSink(new S3Sink("s3://data-lake/orders/")); // Data lake
enriched.addSink(new RedisSink()); // Cache
enriched.addSink(new SnowflakeSink()); // Data warehouse

env.execute("CDC Processing Pipeline");

```

Handling Schema Evolution:

```
// Schema Registry integration
Schema schema = schemaRegistry.getLatestSchema("orders");

// Avro with schema evolution
DataStream<GenericRecord> orders = env
    .addSource(new FlinkKafkaConsumer<>(
        "orders",
        ConfluentRegistryAvroDeserializationSchema.forGeneric(schema,
            schemaRegistryUrl),
        properties
    ));

// Handle backward-compatible changes
// - New optional fields (use defaults)
// - Renamed fields (use aliases)
// - Removed fields (ignore)
```

Pros:

- Near real-time (second-level latency)
- Captures all changes automatically
- No impact on source database performance
- Event-driven architecture

Cons:

- Requires database configuration (binlog/WAL)
- Schema evolution complexity
- Operational overhead (monitoring CDC lag)

4. Batch vs Streaming vs Real-Time {#batch-vs-streaming}

Comparison Matrix

Aspect	Batch	Streaming	Real-Time
Latency	Hours to days	Seconds to minutes	Milliseconds
Data Window	Full dataset	Sliding/tumbling windows	Single event
Complexity	Low	Medium	High
Cost	Low	Medium	High
Use Cases	Reports, analytics	Monitoring, metrics	Trading, fraud detection
Technologies	Spark, Airflow	Kafka, Flink	Custom, in-memory
Throughput	Very high	High	Medium

Aspect	Batch	Streaming	Real-Time
State Management	Not needed	Checkpointing	In-memory

When to Use Each

Batch Processing:

- ✓ Use When:
 - Daily/weekly reports
 - Historical analysis
 - Large-scale data transformations
 - Cost is primary concern
 - Latency > 1 hour is acceptable
- X Don't Use When:
 - Need real-time insights
 - Time-sensitive decisions
 - Continuous monitoring required

Stream Processing:

- ✓ Use When:
 - Real-time dashboards
 - Alerting and monitoring
 - Continuous aggregations
 - Event-driven workflows
 - Latency: seconds to minutes
- X Don't Use When:
 - Simple periodic jobs (use batch)
 - Ultra low-latency required (<100ms)
 - Complex multi-table joins (consider batch)

Real-Time Processing:

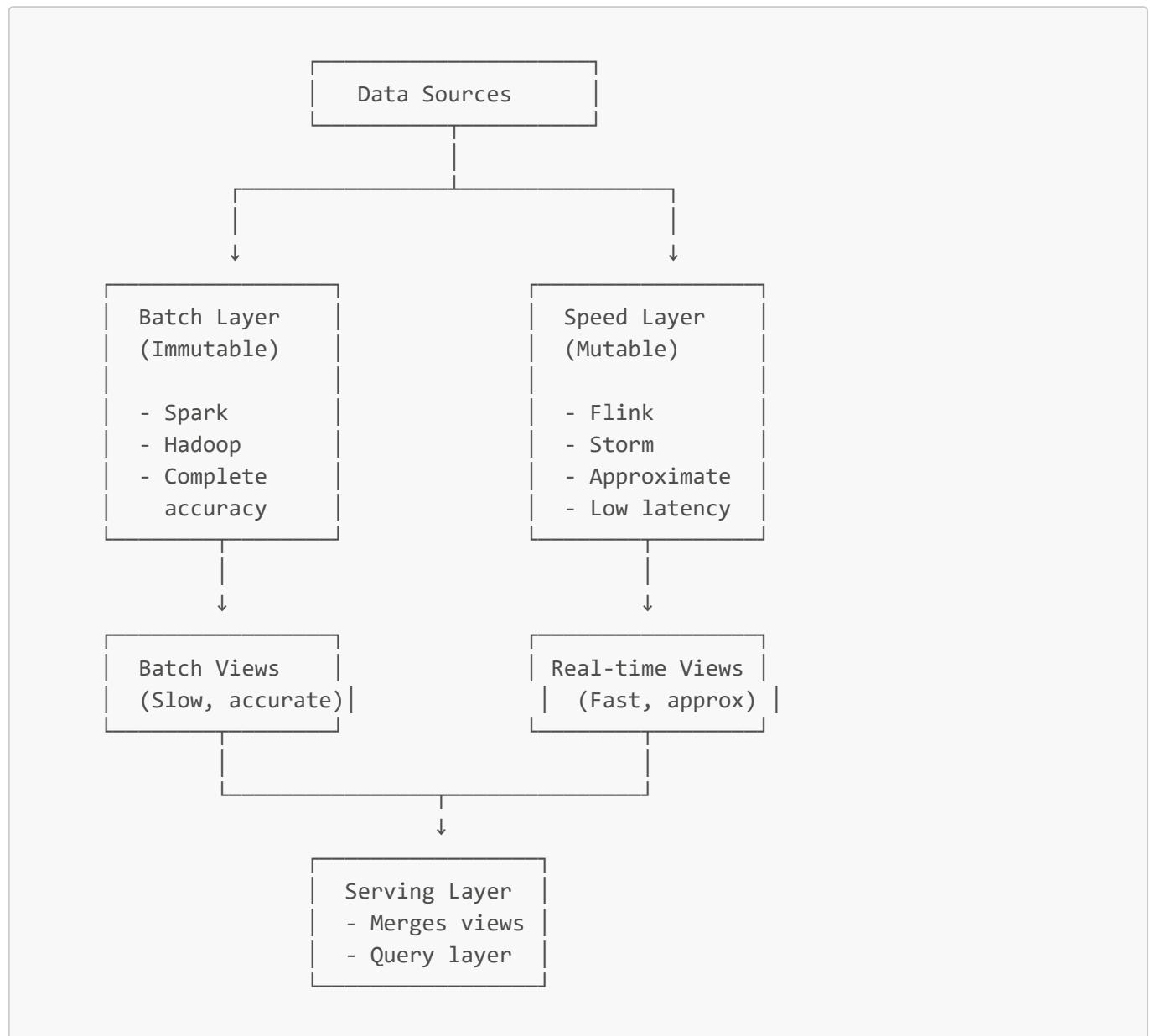
- ✓ Use When:
 - High-frequency trading
 - Fraud detection
 - Real-time bidding
 - Gaming leaderboards
 - Latency < 100ms required
- X Don't Use When:
 - Cost is primary concern
 - Simple use case (over-engineering)
 - Team lacks expertise

5. Lambda Architecture {#lambda-architecture}

Overview

Lambda architecture combines batch and stream processing to provide both accurate historical views and fast real-time approximations.

Architecture Diagram:



Components

1. Batch Layer:

```
# Spark batch job (runs nightly)
def compute_batch_views(date):
    events = spark.read.parquet(f"s3://raw/events/date={date}")

    # Compute accurate aggregations
```

```

user_metrics = events.groupBy("user_id").agg(
    count("*").alias("event_count"),
    sum("amount").alias("total_amount"),
    countDistinct("session_id").alias("sessions")
)

# Write to batch views
user_metrics.write.mode("overwrite") \
    .parquet(f"s3://batch-views/user-metrics/date={date}")

# Schedule with Airflow
@dag(schedule_interval='@daily')
def batch_pipeline():
    compute_batch = PythonOperator(
        task_id='compute_batch_views',
        python_callable=compute_batch_views
    )

```

2. Speed Layer:

```

// Flink streaming job (runs continuously)
public class SpeedLayer {
    public static void main(String[] args) throws Exception {
        StreamExecutionEnvironment env =
            StreamExecutionEnvironment.getExecutionEnvironment();

        DataStream<Event> events = env.addSource(new KafkaSource());

        // Compute approximate aggregations (last N hours)
        DataStream<UserMetrics> realtimeMetrics = events
            .keyBy(event -> event.getUserId())
            .window(TumblingEventTimeWindows.of(Time.hours(1)))
            .aggregate(new MetricsAggregator());

        // Write to speed views
        realtimeMetrics.addSink(new RedisSink());

        env.execute("Speed Layer");
    }
}

```

3. Serving Layer:

```

# Query merges batch and speed views
def get_user_metrics(user_id, start_date, end_date):
    # Get batch views (historical, complete)
    batch_metrics = query_batch_views(user_id, start_date, end_date)

    # Get speed views (recent, approximate)

```

```
realtime_metrics = query_speed_views(user_id)

# Merge
total_events = batch_metrics['event_count'] + realtime_metrics['event_count']
total_amount = batch_metrics['total_amount'] +
realtime_metrics['total_amount']

return {
    'user_id': user_id,
    'total_events': total_events,
    'total_amount': total_amount,
    'last_updated': realtime_metrics['timestamp']
}
```

Pros and Cons

Pros:

- **Accuracy:** Batch layer provides exact results
- **Low Latency:** Speed layer provides fast approximations
- **Fault Tolerance:** Immutable batch layer is reliable
- **Flexibility:** Can optimize each layer independently

Cons:

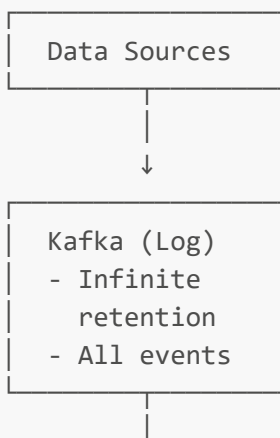
- **Complexity:** Maintain two separate codebases
- **Consistency:** Merging views can be tricky
- **Cost:** Running both layers simultaneously
- **Development:** Need expertise in both batch and streaming

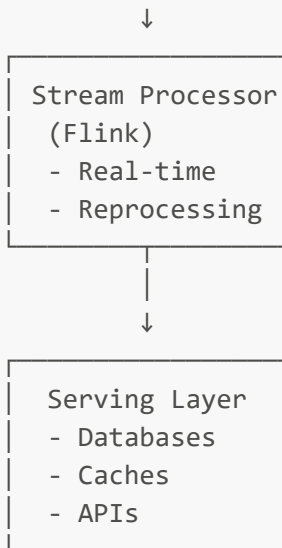
6. Kappa Architecture {#kappa-architecture}

Overview

Kappa architecture simplifies Lambda by using **only stream processing** for both real-time and historical data.

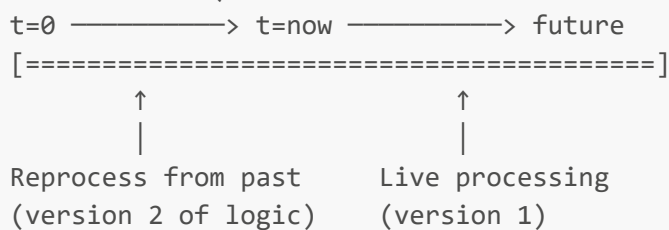
Architecture Diagram:





Key Concept: Reprocessing

Event Stream (Kafka with infinite retention):



Example: Reprocessing with Flink:

```
// Version 1: Original logic (deployed Week 1)
DataStream<Metrics> v1 = events
    .keyBy(e -> e.userId)
    .window(TumblingEventTimeWindows.of(Time.days(1)))
    .aggregate(new SimpleAggregator());

// Week 3: Bug found in SimpleAggregator
// Deploy Version 2 with fixed logic

// Version 2: New job processes from beginning
env.setRestartStrategy(RestartStrategies.noRestart());

DataStream<Metrics> v2 = events
    .assignTimestampsAndWatermarks(WatermarkStrategy
        .forMonotonousTimestamps())
    .keyBy(e -> e.userId)
    .window(TumblingEventTimeWindows.of(Time.days(1)))
    .aggregate(new FixedAggregator());

// Kafka offset reset to beginning
properties.setProperty("auto.offset.reset", "earliest");
```

```
// Reprocess all historical data with new logic
// Once caught up, v2 replaces v1 for live processing
```

Pros and Cons

Pros:

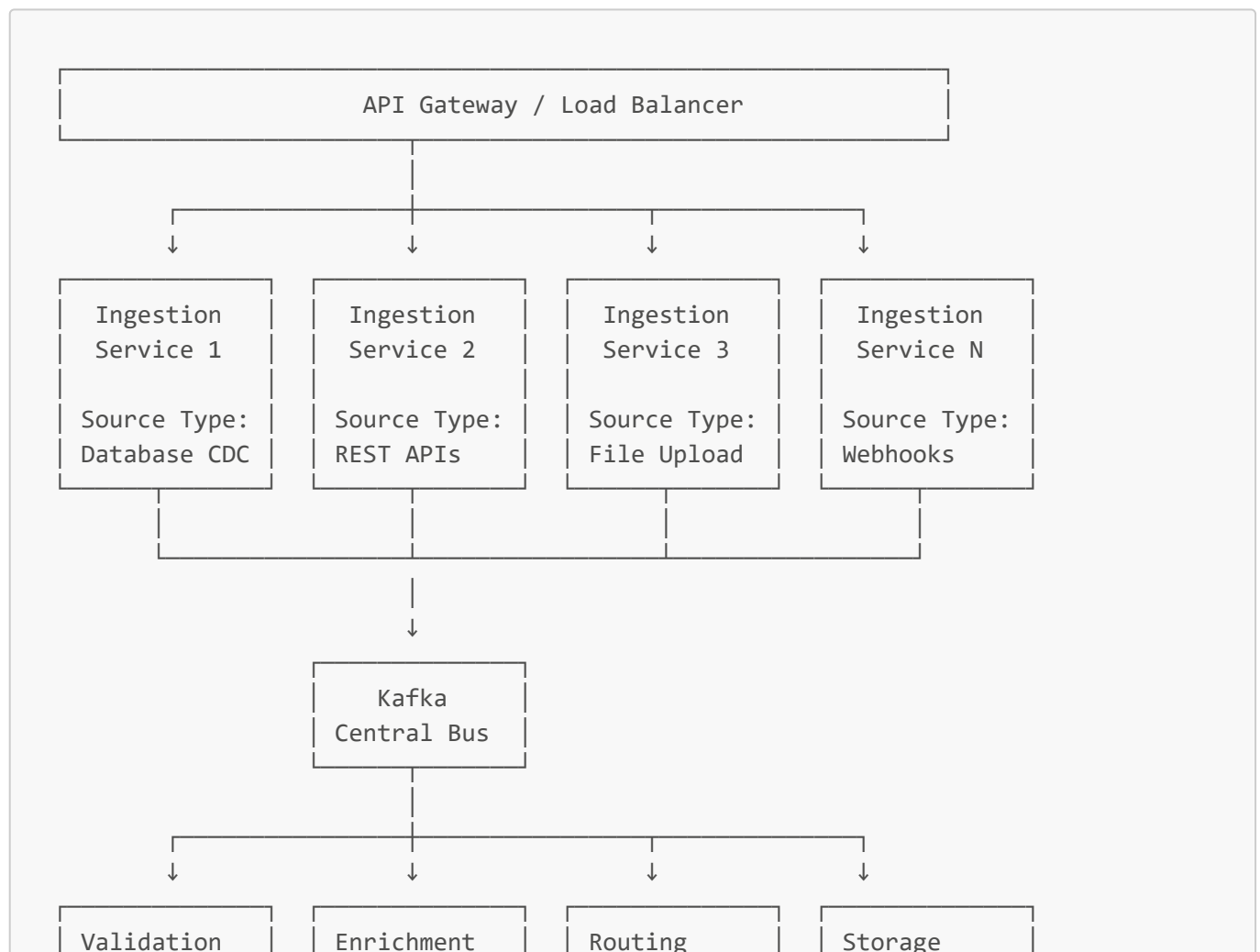
- **Simplicity:** Single codebase for all processing
- **Consistency:** No view merging needed
- **Flexibility:** Easy to reprocess with new logic
- **Lower Maintenance:** One system to operate

Cons:

- **Storage:** Kafka retention can be expensive
- **Reprocessing Time:** Can take hours/days for large datasets
- **Resource Intensive:** Stream processing for everything
- **Complexity:** Stream processing harder than batch for some use cases

7. Microservices-Based Ingestion {#microservices-ingestion}

Architecture



Service

Service

Service

Service

Example: Database CDC Microservice

```
# FastAPI-based CDC ingestion service
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import asyncio
from kafka import KafkaProducer
import json

app = FastAPI(title="CDC Ingestion Service")

# Kafka producer
producer = KafkaProducer(
    bootstrap_servers='kafka:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
    acks='all',
    retries=3
)

class CDCEvent(BaseModel):
    table: str
    operation: str # INSERT, UPDATE, DELETE
    before: dict
    after: dict
    timestamp: str

@app.post("/ingest/cdc")
async def ingest_cdc(event: CDCEvent):
    """
    Receive CDC events and publish to Kafka
    """
    try:
        # Validate event
        if event.operation not in ['INSERT', 'UPDATE', 'DELETE']:
            raise HTTPException(status_code=400, detail="Invalid operation")

        # Enrich with metadata
        enriched_event = {
            **event.dict(),
            'service': 'cdc-ingestion',
            'version': '1.0'
        }

        # Publish to Kafka topic
        topic = f"cdc.{event.table}"
        future = producer.send(topic, enriched_event)

        # Wait for acknowledgment
```

```

        record_metadata = future.get(timeout=10)

        return {
            'status': 'success',
            'topic': record_metadata.topic,
            'partition': record_metadata.partition,
            'offset': record_metadata.offset
        }

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/health")
async def health():
    """Health check endpoint"""
    return {'status': 'healthy'}

# Metrics endpoint
@app.get("/metrics")
async def metrics():
    return {
        'events_processed': get_events_count(),
        'kafka_lag': get_kafka_lag(),
        'error_rate': get_error_rate()
    }

```

Deployment (Kubernetes):

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: cdc-ingestion-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: cdc-ingestion
  template:
    metadata:
      labels:
        app: cdc-ingestion
    spec:
      containers:
        - name: cdc-ingestion
          image: cdc-ingestion:1.0
          ports:
            - containerPort: 8000
          env:
            - name: KAFKA_BOOTSTRAP_SERVERS
              value: "kafka:9092"
          resources:
            requests:

```

```
    memory: "512Mi"
    cpu: "500m"
  limits:
    memory: "1Gi"
    cpu: "1000m"
  livenessProbe:
    httpGet:
      path: /health
      port: 8000
    initialDelaySeconds: 30
    periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: cdc-ingestion-service
spec:
  selector:
    app: cdc-ingestion
  ports:
    - port: 80
      targetPort: 8000
  type: LoadBalancer
```

Pros and Cons

Pros:

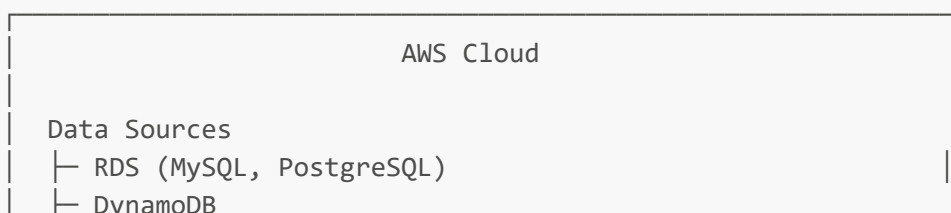
- **Scalability:** Scale services independently
- **Isolation:** Failures don't cascade
- **Flexibility:** Different tech stacks per service
- **Team Autonomy:** Teams own their services

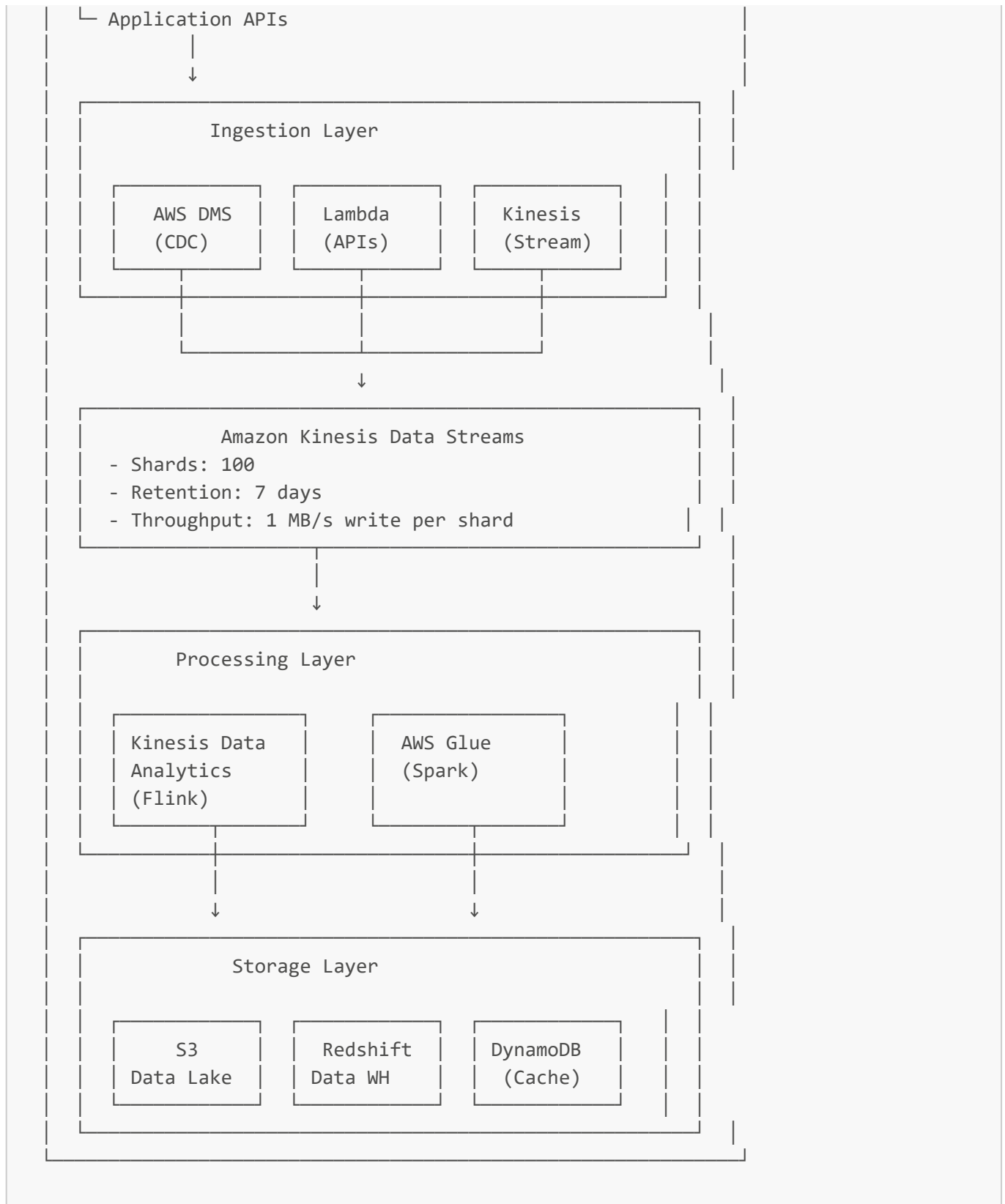
Cons:

- **Complexity:** More moving parts
- **Operational Overhead:** More services to monitor
- **Network Latency:** Inter-service communication
- **Data Consistency:** Distributed transactions

8. Cloud-Native Architectures {#cloud-native}

AWS Architecture





AWS Glue ETL Job Example:

```

import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job

```

```

args = getResolvedOptions(sys.argv, ['JOB_NAME'])
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

# Read from S3 (catalog table)
datasource = glueContext.create_dynamic_frame.from_catalog(
    database="raw_data",
    table_name="events"
)

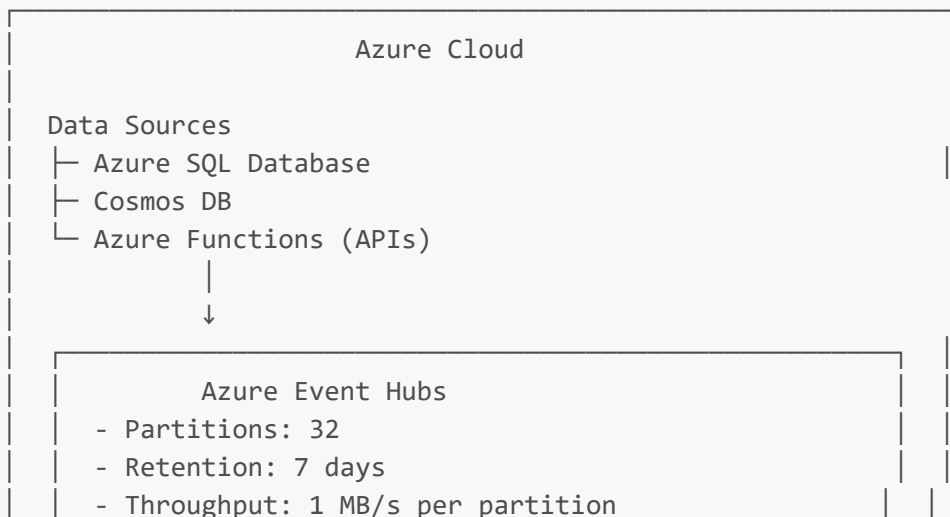
# Transform
transformed = ApplyMapping.apply(
    frame=datasource,
    mappings=[
        ("event_id", "string", "event_id", "string"),
        ("user_id", "string", "user_id", "string"),
        ("event_type", "string", "event_type", "string"),
        ("timestamp", "timestamp", "event_time", "timestamp"),
        ("properties", "string", "properties", "string")
    ]
)

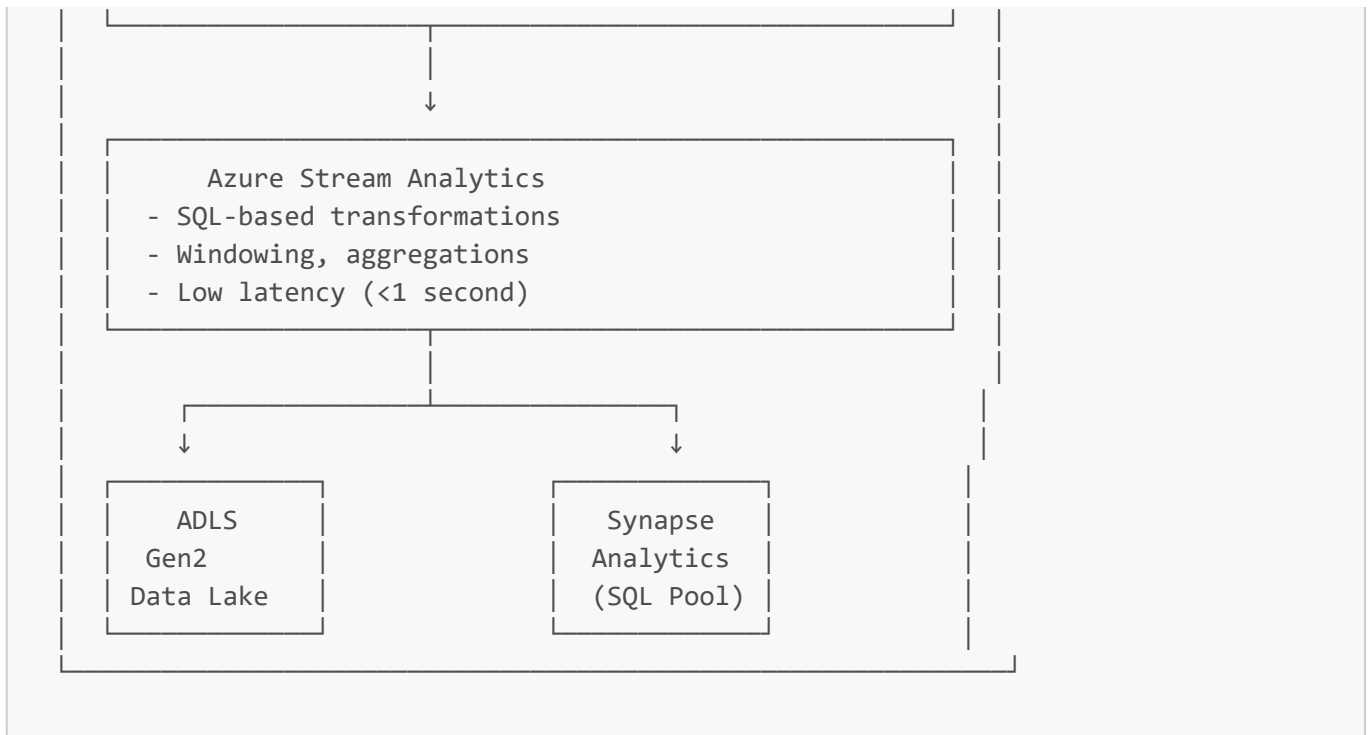
# Write to Redshift
glueContext.write_dynamic_frame.from_catalog(
    frame=transformed,
    database="analytics",
    table_name="processed_events",
    transformation_ctx="write_to_redshift"
)

job.commit()

```

Azure Architecture



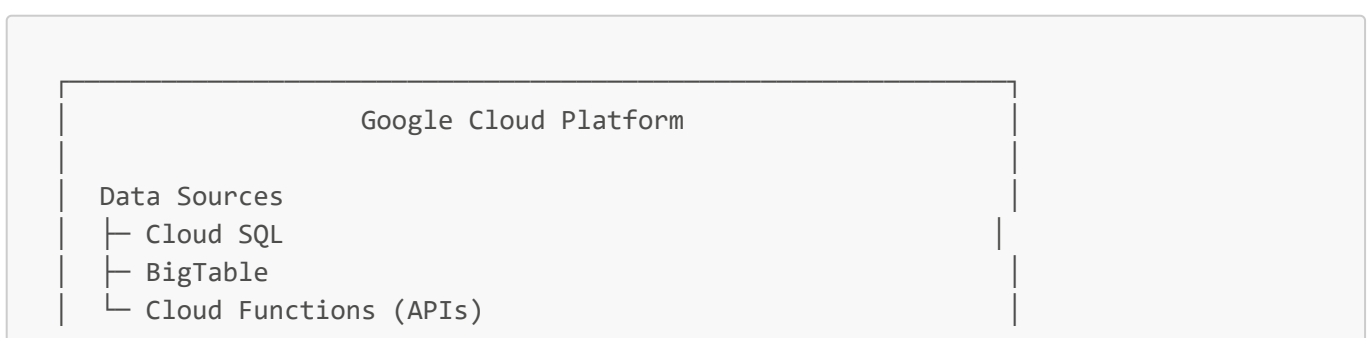


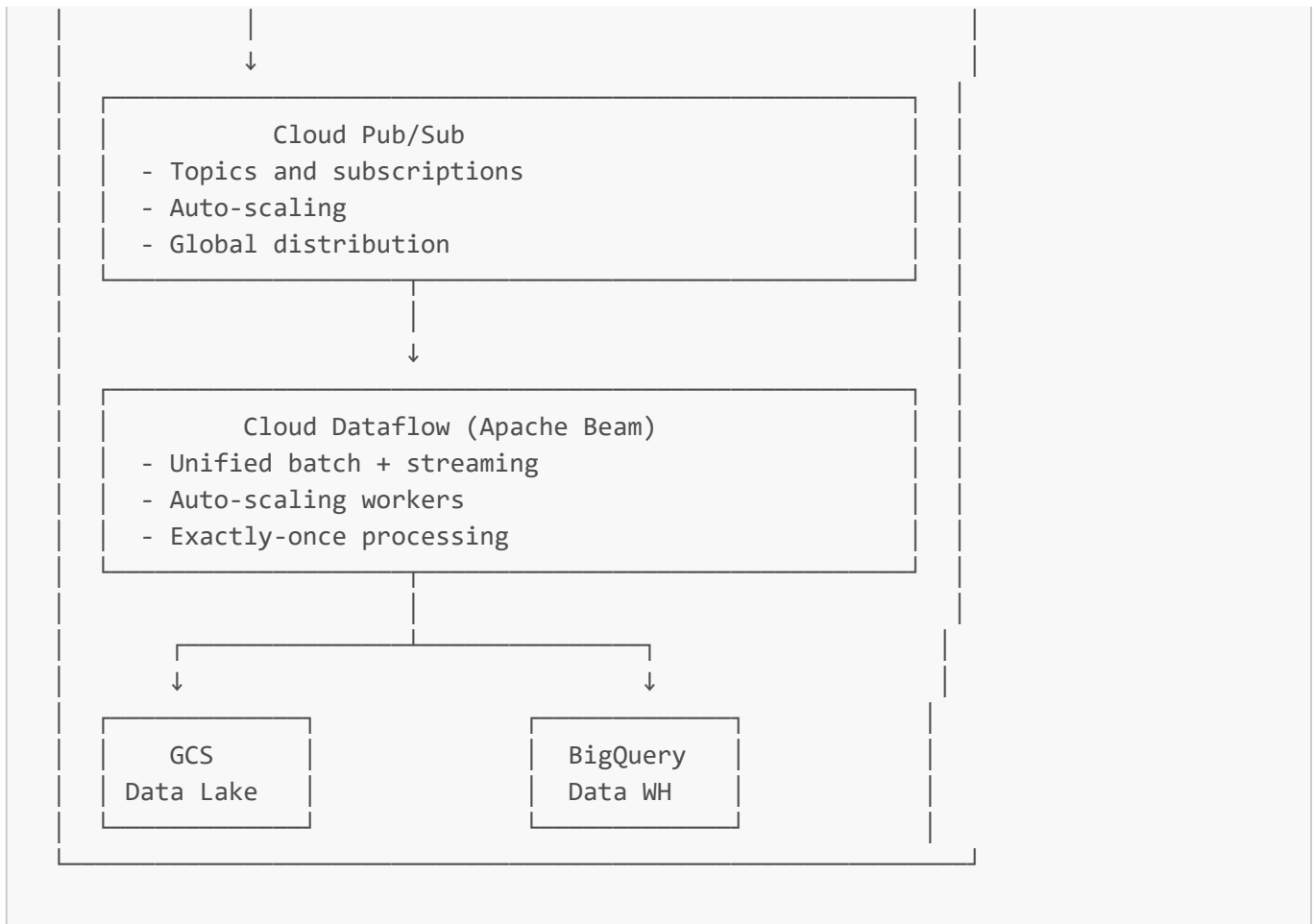
Azure Stream Analytics Query:

```

-- Real-time aggregation with Azure Stream Analytics
SELECT
    user_id,
    event_type,
    System.Timestamp() AS window_end,
    COUNT(*) AS event_count,
    AVG(CAST(properties.amount AS FLOAT)) AS avg_amount
INTO
    [synapse-output]
FROM
    [eventhub-input]
TIMESTAMP BY event_time
GROUP BY
    user_id,
    event_type,
    TumblingWindow(minute, 5)
HAVING
    COUNT(*) > 10
  
```

GCP Architecture





Apache Beam (Dataflow) Example:

```

import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions
from apache_beam.io.gcp.bigquery import WriteToBigQuery

pipeline_options = PipelineOptions(
    project='my-gcp-project',
    runner='DataflowRunner',
    region='us-central1',
    temp_location='gs://my-bucket/temp',
    staging_location='gs://my-bucket/staging'
)

with beam.Pipeline(options=pipeline_options) as pipeline:
    (
        pipeline
        | 'Read from Pub/Sub' >> beam.io.ReadFromPubSub(
            topic='projects/my-project/topics/events'
        )
        | 'Parse JSON' >> beam.Map(lambda x: json.loads(x))
        | 'Extract User Events' >> beam.Map(
            lambda event: {
                'user_id': event['user_id'],
                'event_type': event['event_type'],
                'timestamp': event['timestamp']
            }
        )
    )

```

```

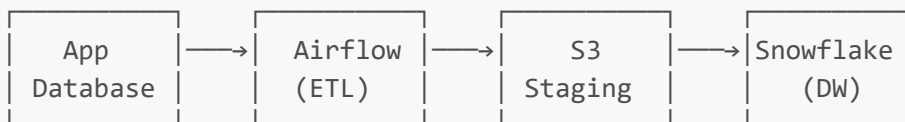
    )
    | 'Window into 5-min intervals' >> beam.WindowInto(
        beam.window.FixedWindows(5 * 60) # 5 minutes
    )
    | 'Count by user' >> beam.combiners.Count.PerKey()
    | 'Write to BigQuery' >> WriteToBigQuery(
        'my-project:dataset.user_events',
        schema='user_id:STRING,event_count:INTEGER',
        create_disposition=beam.io.BigQueryDisposition.CREATE_IF_NEEDED,
        write_disposition=beam.io.BigQueryDisposition.WRITE_APPEND
    )
)
)

```

9. Reference Architectures by Scale {#reference-architectures}

Small Scale (<1M events/day)

Use Case: Startup, single application, limited budget



Components:

- Airflow (1 worker node)
- PostgreSQL (source)
- S3 (staging)
- Snowflake (X-Small warehouse)

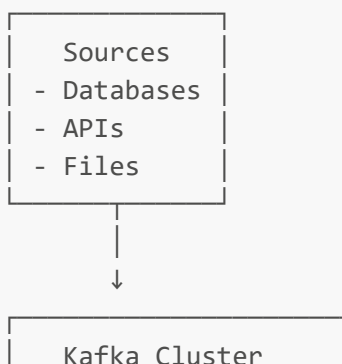
Cost: ~\$500-1K/month

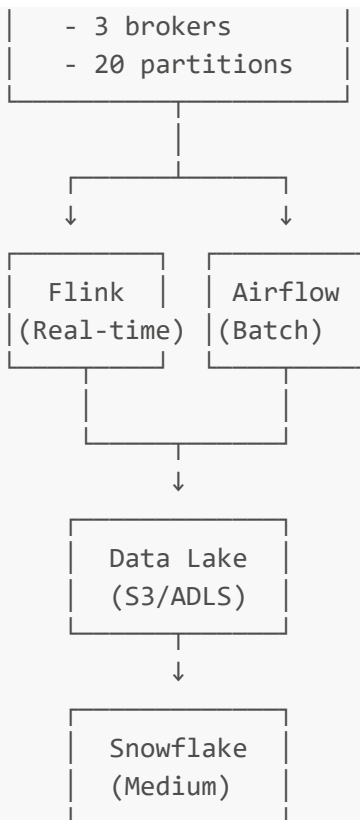
Latency: Hours (batch processing)

Team: 1 data engineer

Medium Scale (1M-100M events/day)

Use Case: Growth company, multiple data sources, real-time needs



**Components:**

- Kafka (3 brokers, 50 GB/day)
- Flink (5 task managers)
- Airflow (3 workers)
- S3 (1 TB storage)
- Snowflake (Medium warehouse)

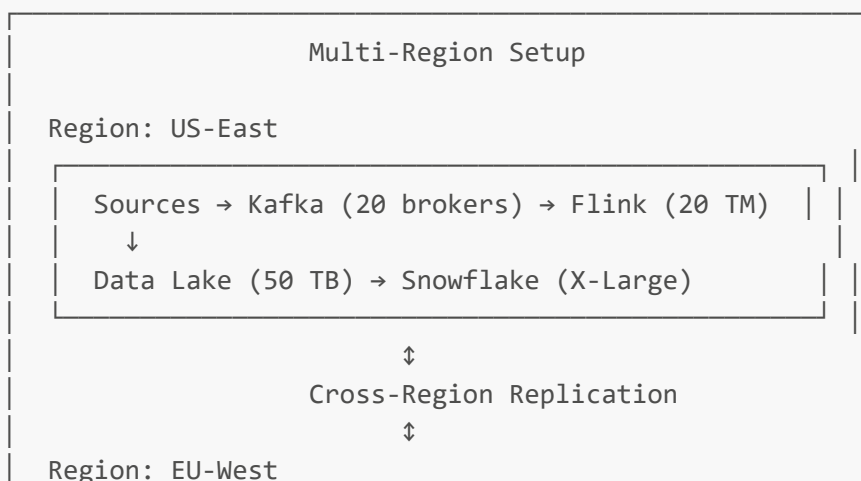
Cost: ~\$5K-15K/month

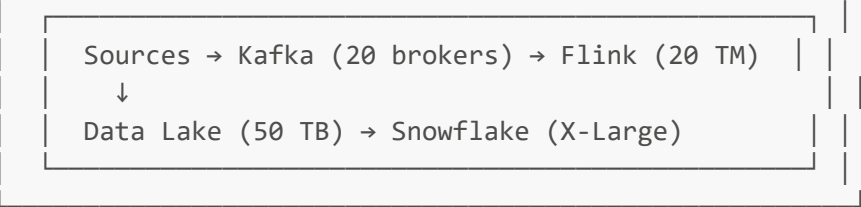
Latency: Seconds (streaming), Hours (batch)

Team: 2-3 data engineers

Large Scale (100M-10B events/day)

Use Case: Enterprise, multiple business units, global operations





```
graph TD; Sources --> Kafka[Kafka (20 brokers)]; Kafka --> Flink[Flink (20 TM)]; Flink --> DL[Data Lake (50 TB)]; DL --> Snowflake[Snowflake (X-Large)];
```

Sources → Kafka (20 brokers) → Flink (20 TM)
↓
Data Lake (50 TB) → Snowflake (X-Large)

Components:

- Kafka (40 brokers, 2 TB/day per region)
- Flink (40 task managers per region)
- S3 (100 TB per region)
- Snowflake (X-Large warehouses)
- Cross-region replication (Kafka MirrorMaker)

Cost: ~\$50K-200K/month

Latency: Sub-second (streaming)

Team: 10+ data engineers

Massive Scale (>10B events/day)

Use Case: Tech giants, social media, IoT platforms

Global Multi-Region Architecture**Region 1 (US-East)**

- └ Kafka Cluster (100+ brokers)
- └ Flink Cluster (200+ task managers)
- └ Data Lake (PB scale)
- └ Multiple Data Warehouses

Region 2 (US-West)

- └ Kafka Cluster (100+ brokers)
- └ Full stack replication

Region 3 (EU)

- └ Kafka Cluster (100+ brokers)
- └ Full stack replication

Region 4 (APAC)

- └ Kafka Cluster (100+ brokers)
- └ Full stack replication

Special Considerations:

- Custom ingestion protocols (not HTTP)
- Custom data formats (highly compressed)
- Edge computing (process at source)
- Tiered storage (hot/warm/cold)
- Custom query engines

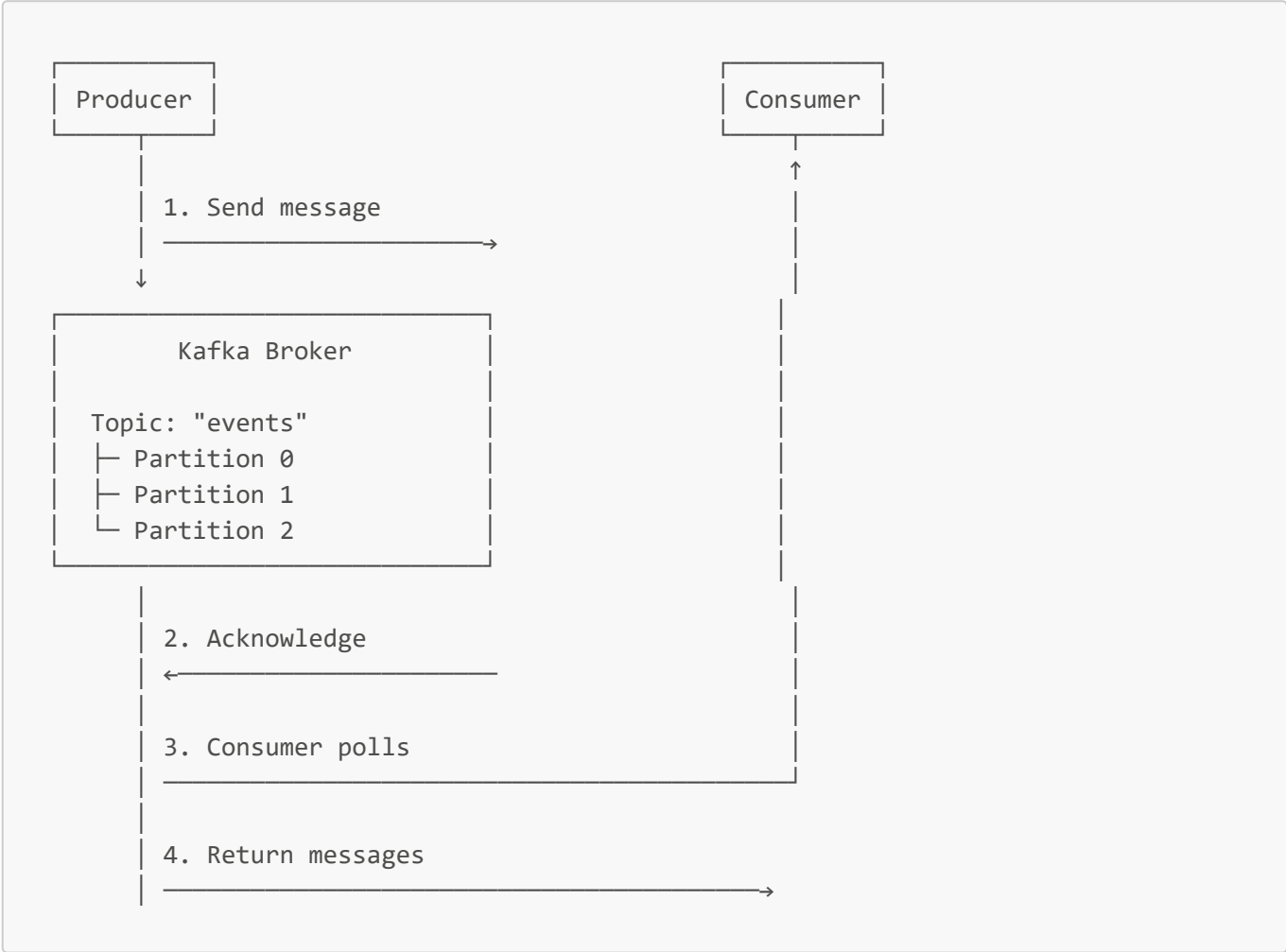
Cost: \$1M+/month

Latency: <100ms (ultra low-latency)
Team: 50+ engineers (dedicated platform team)

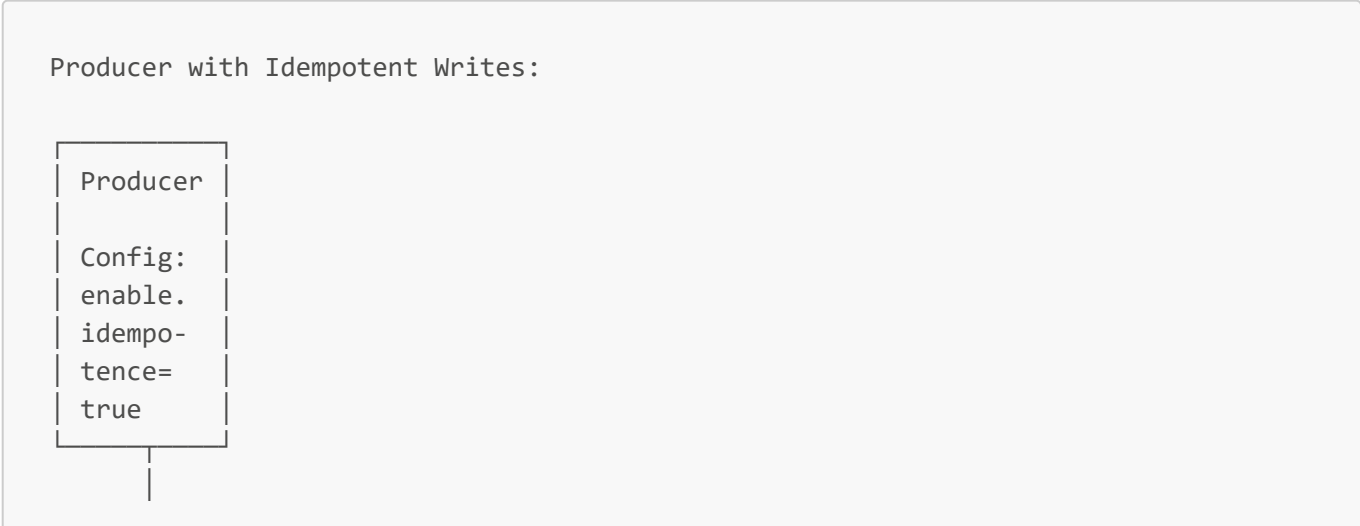
10. Component Interactions {#component-interactions}

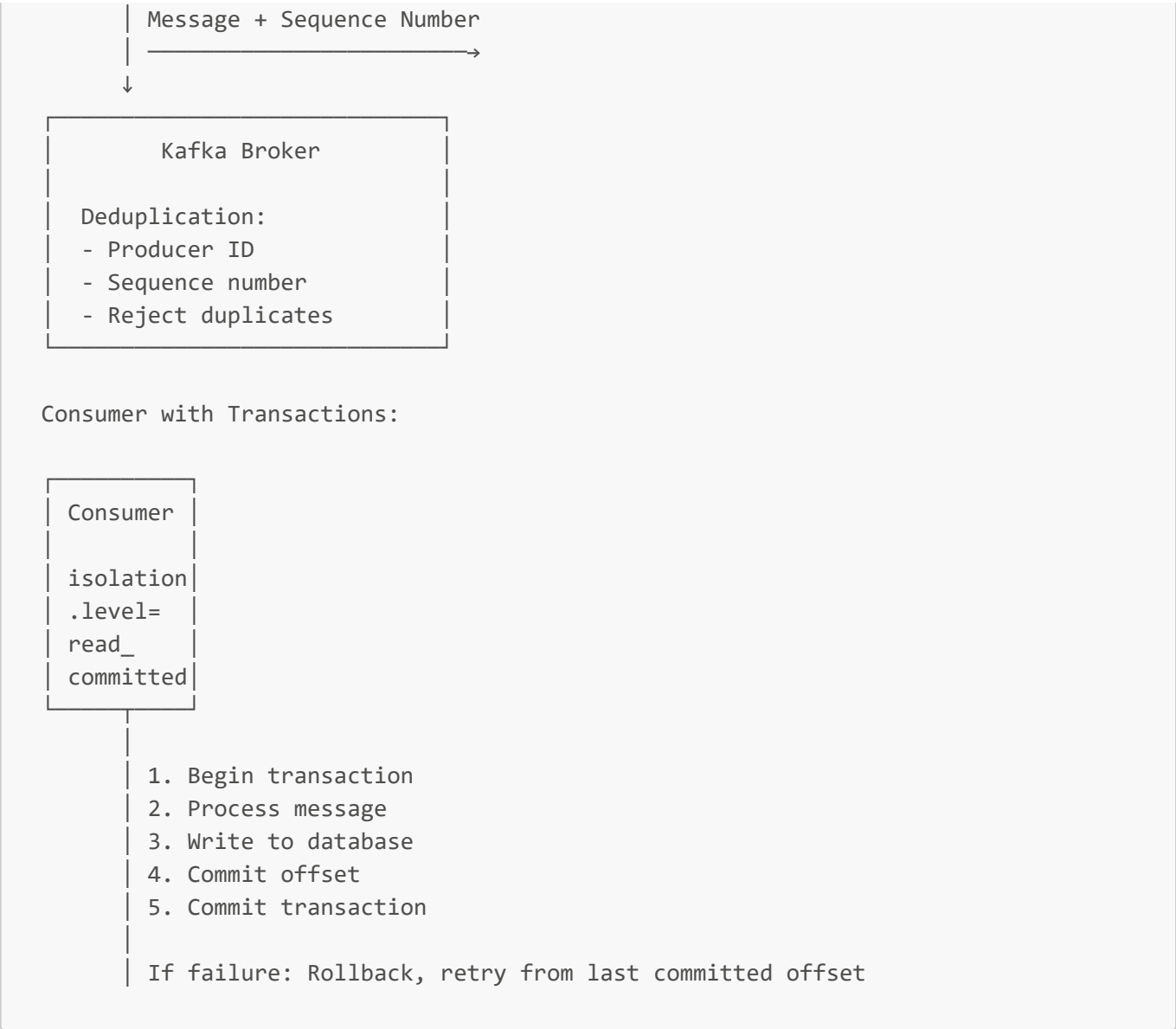
Data Flow Patterns

Pattern 1: Producer → Kafka → Consumer



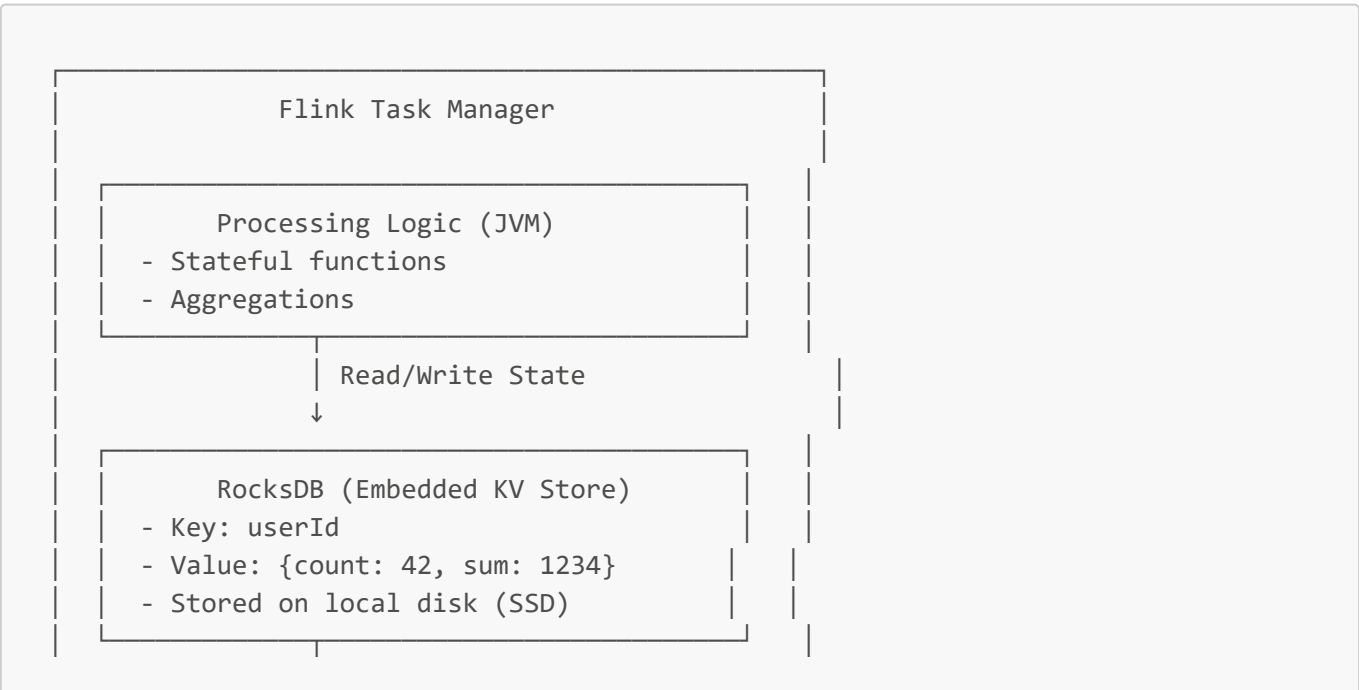
Pattern 2: Exactly-Once Delivery

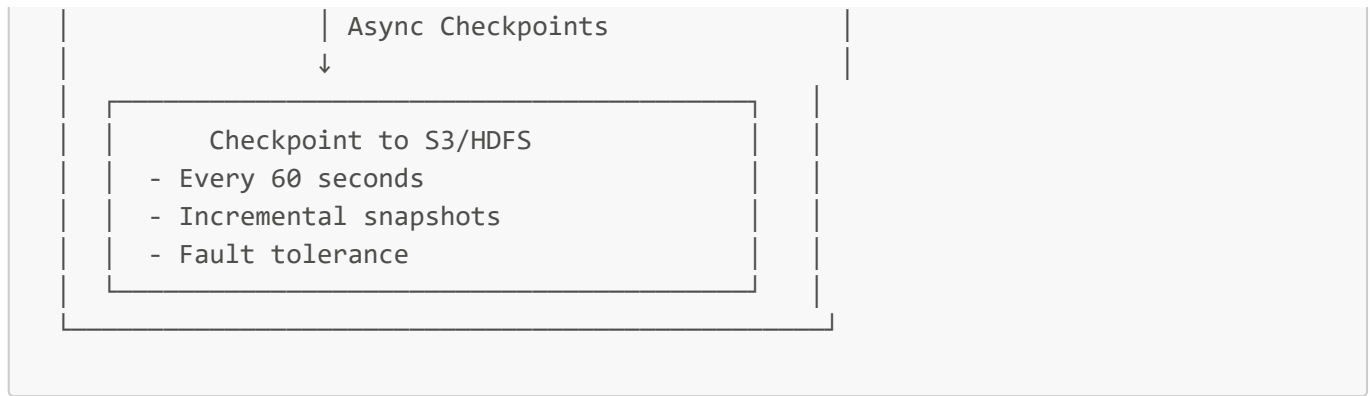




State Management

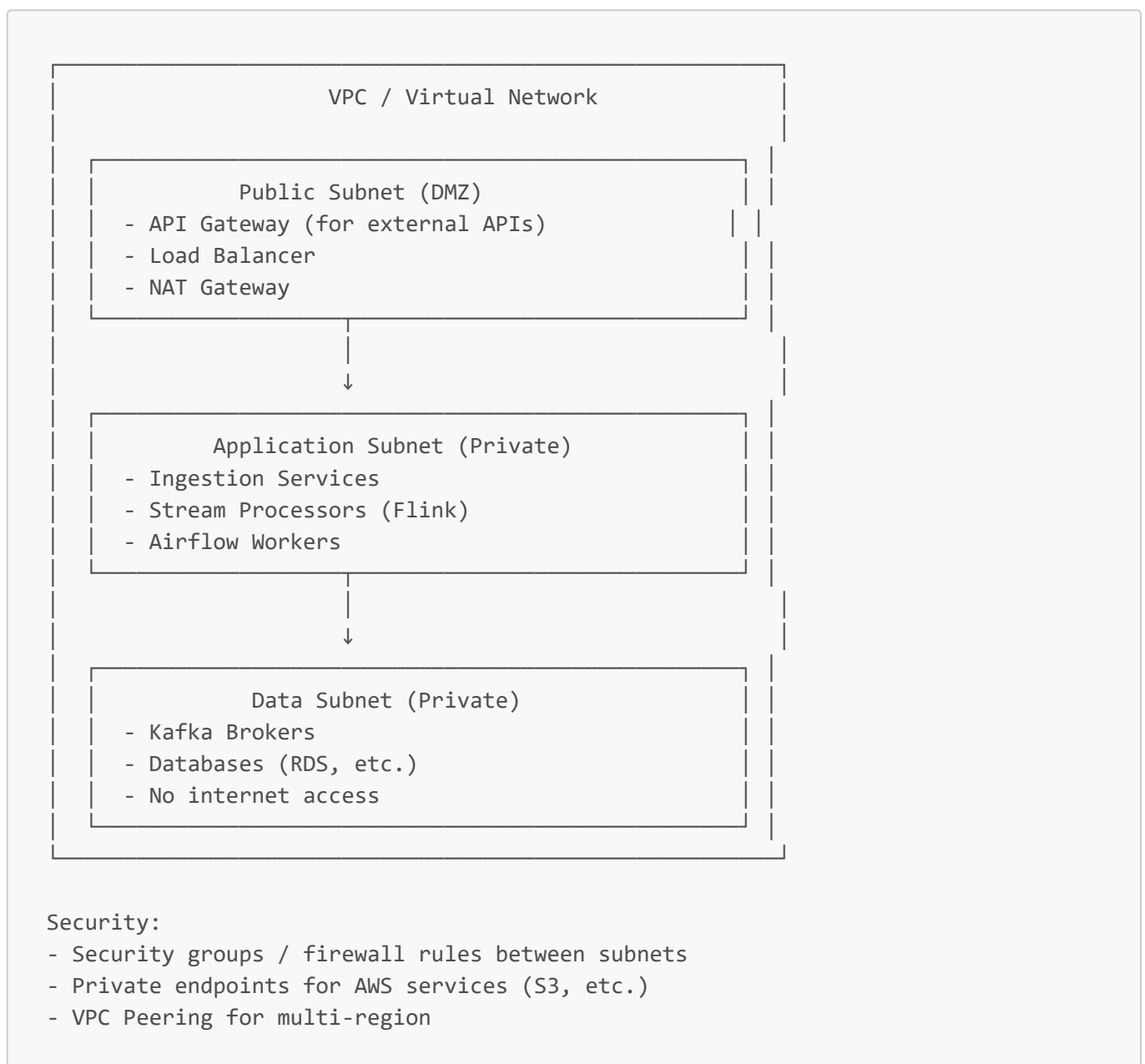
RocksDB in Flink:





11. Network Topology {#network-topology}

Network Segmentation



Bandwidth Considerations

Kafka Cluster Sizing:

Requirements:

- 100 MB/s ingestion rate
- Replication factor: 3
- Retention: 7 days

Calculations:

Network bandwidth per broker = 100 MB/s × 3 (replication) = 300 MB/s

Total network: 300 MB/s × 8 = 2.4 Gbps (need 10 Gbps network)

Storage per broker: 100 MB/s × 86400 s/day × 7 days / 3 brokers = 20 TB

12. High Availability & Fault Tolerance {#ha-fault-tolerance}

Kafka High Availability

Replication & Leadership:

Topic: "transactions" (3 partitions, replication factor = 3)

Partition 0:

```

Leader: Broker 1
Follower: Broker 2
Follower: Broker 3
  
```

┌─┐ In-Sync Replicas (ISR)

Failure Scenario:

1. Broker 1 fails (leader for Partition 0)
2. ZooKeeper detects failure (heartbeat timeout)
3. New leader elected from ISR (Broker 2)
4. Producers/consumers transparently reconnect to Broker 2
5. When Broker 1 recovers, it becomes follower

Configuration:

- min.insync.replicas = 2 (need 2 replicas to acknowledge)
- acks = all (wait for all ISRs before acknowledgment)

Producer Idempotence:

```

Properties props = new Properties();
props.put("bootstrap.servers", "kafka:9092");
props.put("enable.idempotence", "true"); // Exactly-once semantics
props.put("acks", "all");                // Wait for all replicas
props.put("retries", Integer.MAX_VALUE); // Infinite retries
props.put("max.in.flight.requests.per.connection", 5);

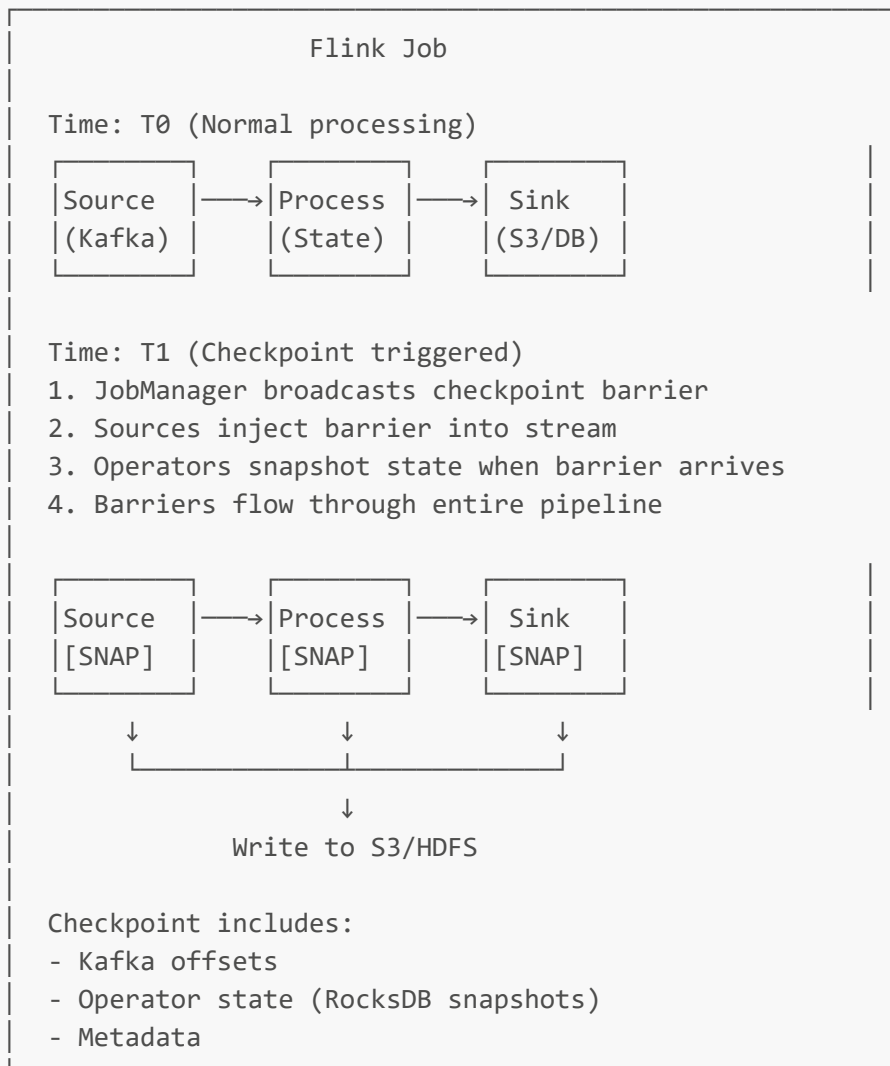
KafkaProducer<String, String> producer = new KafkaProducer<>(props);

// Producer automatically handles:
// - Duplicate message detection (via sequence numbers)
  
```

```
// - Automatic retries on failure
// - In-order delivery per partition
```

Flink Checkpointing

Checkpoint Mechanism:



Recovery:

If failure occurs:

1. Job restarts
2. Loads latest checkpoint
3. Resets Kafka offsets to checkpoint values
4. Restores operator state
5. Resumes processing

Configuration:

```
StreamExecutionEnvironment env =
    StreamExecutionEnvironment.getExecutionEnvironment();
```

```
// Checkpoint every 60 seconds
env.enableCheckpointing(60000);

// Exactly-once mode
env.getCheckpointConfig().setCheckpointingMode(
    CheckpointingMode.EXACTLY_ONCE
);

// Min time between checkpoints
env.getCheckpointConfig().setMinPauseBetweenCheckpoints(30000);

// Checkpoint timeout
env.getCheckpointConfig().setCheckpointTimeout(600000);

// Allow 3 concurrent checkpoints
env.getCheckpointConfig().setMaxConcurrentCheckpoints(1);

// Keep checkpoints on job cancellation
env.getCheckpointConfig().enableExternalizedCheckpoints(
    ExternalizedCheckpointCleanup.RETAIN_ON_CANCELLATION
);
```

Data Lake Redundancy

S3 Durability:

```
AWS S3:
- Durability: 99.999999999% (11 nines)
- Automatically replicated across multiple AZs
- Versioning (keep multiple versions)
- Cross-region replication (for DR)

Configuration:
aws s3api put-bucket-versioning \
  --bucket my-data-lake \
  --versioning-configuration Status=Enabled

aws s3api put-bucket-replication \
  --bucket my-data-lake \
  --replication-configuration file://replication.json

# replication.json
{
  "Role": "arn:aws:iam::account:role/replication-role",
  "Rules": [{
    "Status": "Enabled",
    "Priority": 1,
    "Filter": {},
    "Destination": {
      "Bucket": "arn:aws:s3::my-data-lake-dr",
```

```
    "ReplicationTime": {  
      "Status": "Enabled",  
      "Time": {  
        "Minutes": 15  
      }  
    }  
  }  
}  
}]  
}
```

Conclusion

This comprehensive architecture overview covers the fundamental building blocks of modern data ingestion systems. Each layer, pattern, and architecture has specific trade-offs and use cases.

Key Takeaways:

1. **Start Simple:** Don't over-engineer. Begin with batch processing if latency isn't critical.
2. **Scale Gradually:** Move from batch → micro-batch → streaming as needs evolve.
3. **Choose the Right Pattern:** Lambda for accuracy + speed, Kappa for simplicity.
4. **Cloud-Native When Possible:** Managed services reduce operational burden.
5. **Plan for Failure:** Design for high availability from day one.
6. **Monitor Everything:** Observability is critical for data pipelines.

Next Steps:

- Explore [Design Patterns](#) for specific implementation patterns
- Review [Technology Stack](#) for tool selection
- Study [Case Studies](#) for real-world examples

Version: 1.0